

AI534 — Written Homework Assignment 3 — Manogna

1. (Naive Bayes Classifier) (7 pts) Consider the following training set:

A	B	C	Y
0	1	1	0
1	1	1	0
0	0	0	0
1	1	0	1
0	1	0	1
1	0	1	1

- (a) (3 pts) Learn a Naive Bayes classifier by estimating all necessary probabilities (there should be 7 independent probabilities to be estimated in total).

The generative process is as follows.

Learn the prior probability: for $j = 1, \dots, k$, where $k = 2$

$$P(y = y_1 = 0) = 0.5, \quad P(y = y_2 = 1) = 0.5$$

Learn the probability distribution on $\mathbf{x}$ conditionally independent on each y_j for $i = 1, \dots, d$, where $d = 3$ in the set $\mathbf{x}_1 = A, \mathbf{x}_2 = B, \mathbf{x}_3 = C$

$$P(\mathbf{x}_1 = 1 | y_1) = \frac{1}{3}, \quad P(\mathbf{x}_1 = 1 | y_2) = \frac{2}{3}$$

$$P(\mathbf{x}_2 = 0 | y_1) = \frac{1}{3}, \quad P(\mathbf{x}_2 = 0 | y_2) = \frac{1}{3}$$

$$P(\mathbf{x}_3 = 1 | y_1) = \frac{1}{3}, \quad P(\mathbf{x}_3 = 1 | y_2) = \frac{2}{3}$$

- (b) (3 pts) Compute the probability $P(y = 1 | A = 1, B = 0, C = 0)$.

Computing $P(y | \mathbf{x} = (1, 0, 0))$ using the Naive Bayes formula based on conditional independence we get

$$P(y = 1 | \mathbf{x} = (1, 0, 0)) = \frac{\left(\frac{2}{3}\frac{1}{3}\frac{2}{3}\right)\frac{1}{2}}{\left[\left(\frac{2}{3}\frac{1}{3}\frac{2}{3}\right)\frac{1}{2} + \left(\frac{1}{3}\frac{1}{3}\frac{1}{3}\right)\frac{1}{2}\right]} = \frac{\frac{4}{54}}{\frac{5}{54}} = \frac{4}{5}$$

Similarly,

$$P(y = 0 | \mathbf{x} = (1, 0, 0)) = 1 - \frac{4}{5} = \frac{1}{5}$$

Since $P(y = 1 | \mathbf{x} = (1, 0, 0)) > P(y = 0 | \mathbf{x} = (1, 0, 0))$, then we can predict $P(y = 1 | \mathbf{x} = (1, 0, 0))$

- (c) (1 pts) Suppose we know that the three features A, B and C are independent from one another, can we say that the Naive Bayes assumption is valid? (Note that the particular data set is irrelevant for this question). If your answer is yes, please explain why; if your answer is no please give an counter example.

No, if we know that A, B and C are independent random variables, then the Naive Bayes's assumption is not valid. This is because independence does not imply conditional independence, which is assumed by the Naive Bayes classifier.

As a counter-example, see that for independence $P(x = (A, B, C)) = P(A)P(B)P(C)$ while for conditional independence $P(x = (A, B, C) | y) = P(A|y)P(B|y)P(C|y)$

2. (Naive Bayes learns linear decision boundary.) (10 pts) Show that the following naive Bayes classifiers learn linear decision boundary $w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d = 0$. Express the weights using the corresponding Naive Bayes parameters. Hint: start with the decision boundary defined by $\log \frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})} = 0$.

- (a) Bernoulli Naive Bayes model, where features $x_1, x_2, \dots, x_d$ are binary indicating the presence/absence of words in the vocabulary.

To show that Naive Bayes classifiers learn a linear decision boundary, we start by expressing the decision boundary condition as:

$$\log \frac{P(y=1|x)}{P(y=0|x)} = 0 \implies P(y=1|x) = P(y=0|x).$$

Using Bayes' theorem:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)},$$

$$\frac{P(y=1|x)}{P(y=0|x)} = \frac{P(x|y=1)P(y=1)}{P(x|y=0)P(y=0)}$$

$$\log \frac{P(y=1|x)}{P(y=0|x)} = \log P(y=1) - \log P(y=0) + \log P(x|y=1) - \log P(x|y=0)$$

In this model, features $x_1, x_2, \dots, x_d$ are binary, where the presence ($x_i = 1$) or absence ($x_i = 0$) of words. The conditional probabilities are parameterized as:

$$P(x|y) = \prod_{i=1}^d P(x_i|y).$$

conditional likelihoods for $y = 1$ and $y = 0$ are:

$$P(x|y=1) = \prod_{i=1}^d p_i^{x_i} (1-p_i)^{1-x_i}, \quad P(x|y=0) = \prod_{i=1}^d q_i^{x_i} (1-q_i)^{1-x_i},$$

where $p_i = P(x_i = 1|y=1)$ and $q_i = P(x_i = 1|y=0)$

Substitute into the decision boundary:

$$\log \frac{P(y=1)}{P(y=0)} + \sum_{i=1}^d \left[x_i \log \frac{p_i}{q_i} + (1-x_i) \log \frac{1-p_i}{1-q_i} \right] = 0.$$

$$\log \frac{P(y=1)}{P(y=0)} + \sum_{i=1}^d x_i \log \frac{p_i(1-q_i)}{q_i(1-p_i)} + \sum_{i=1}^d \log \frac{1-p_i}{1-q_i} = 0.$$

Combine terms into a linear form:

$$w_0 + \sum_{i=1}^d w_i x_i = 0,$$

$$w_0 = \log \frac{P(y=1)}{P(y=0)} + \sum_{i=1}^d \log \frac{1-p_i}{1-q_i}, \quad w_i = \log \frac{p_i(1-q_i)}{q_i(1-p_i)}.$$

- (b) Multinomial Naive Bayes Model, where $x_1, \dots, x_d$ representing counts of words $w_1, \dots, w_d$ in the vocabulary. Express the weights using the Naive Bayes parameters: the class priors $P(y=1), P(y=0)$ and the class conditionals: $p(w_i|y=1)$ and $p(w_i|y=0)$

Here $x_1, x_2, \dots, x_d$ represent counts of words $w_1, w_2, \dots, w_d$

$$P(x | y) = \frac{(\sum_{i=1}^d x_i)!}{x_1! x_2! \cdots x_d!} \prod_{i=1}^d P(w_i | y)^{x_i}$$

$$P(x | y = 1) = \prod_{i=1}^d p_i^{x_i}, \quad P(x | y = 0) = \prod_{i=1}^d q_i^{x_i}$$

$$\log \frac{P(y = 1)}{P(y = 0)} + \sum_{i=1}^d x_i \log \frac{p_i}{q_i} = 0$$

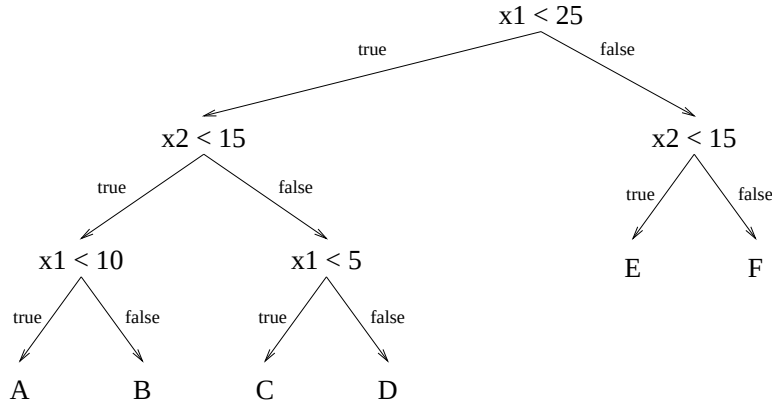
$$w_0 + \sum_{i=1}^d w_i x_i = 0,$$

$$w_0 = \log \frac{P(y = 1)}{P(y = 0)}, \quad w_i = \log \frac{p_i}{q_i}.$$

Both Bernoulli and Multinomial Naïve Bayes learn linear decision boundaries. The weights w_0, w_i are shown using the class priors and conditional probabilities:

- For Bernoulli: $w_0 = \log \frac{P(y=1)}{P(y=0)} + \sum_{i=1}^d \log \frac{1-p_i}{1-q_i}$, $w_i = \log \frac{p_i(1-q_i)}{q_i(1-p_i)}$.
- For Multinomial: $w_0 = \log \frac{P(y=1)}{P(y=0)}$, $w_i = \log \frac{p_i}{q_i}$.

3. (6 pts) Consider the following decision tree:



- (a) (2 pts) Draw the decision boundaries defined by this tree. Each leaf of the tree is labeled with a letter. Write this letter in the corresponding region of input space.

To draw the decision boundaries defined by the tree:

The splits occur based on x_1 and x_2 values. The splits are: $x_1 < 25$, - $x_2 < 15$, $x_1 < 10$, $x_1 < 5$
 Each path through the tree corresponds to a region in the input space.
 Each leaf node corresponds to a specific region A, B, C, D, E, F.

- (b) (2 pts) Give another decision tree that is syntactically different but defines the same decision boundaries. This demonstrates that the space of decision trees is syntactically redundant.

To provide another decision tree that is syntactically different but defines the same decision boundaries, we can rearrange the sequence of splits while ensuring the overall partitioning of the input space remains unchanged.

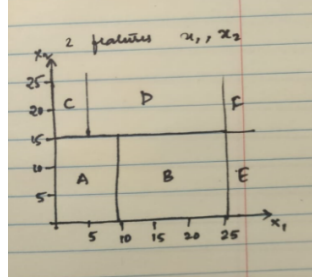


Figure 1: Decision Tree boundaries

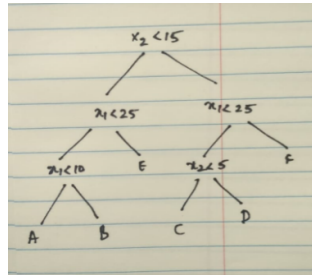


Figure 2: 3(b) Decision Tree

The alternative tree is syntactically different because the splits are ordered differently ($x_2 < 15$ is evaluated first instead of $x_1 < 25$) but the regions A, B, C, D, E, F are defined exactly the same in the input space, showing redundancy in the representation of decision trees.

- (c) (2pts) How does this redundancy influence learning (does it make it easier or harder to find an accurate tree)?

Redundancy influences learning as follows:

Harder to Learn: Redundancy increases the search space of potential trees. Since multiple trees can represent the same decision boundaries, the learning algorithm might need more time or struggle to find the simplest, most accurate tree. This can lead to overfitting or inefficiency.

Improvement with Pruning or Ensembles: While redundancy complicates standard decision tree learning, techniques like pruning or ensemble learning (e.g., random forests) can exploit this redundancy. Pruning simplifies the tree by removing unnecessary splits, while ensembles leverage diverse representations to improve accuracy and robustness.

Redundancy can be a challenge for individual tree learning but an advantage when managed well in advanced techniques.

4. (6 pts) In the basic decision tree algorithm (assuming we always create binary splits), we choose the feature/value pair with the maximum information gain as the test to use at each internal node of the decision tree. Suppose we modified the algorithm to choose at random from among those feature/value combinations that had non-zero mutual information, and we kept all other parts of the algorithm unchanged.

- (a) (2 pts) What is the maximum number of leaf nodes that such a decision tree could contain if it were trained on m training examples?

The maximum number of leaf nodes stays the same as with the normal method. It doesn't matter how we pick the splits (randomly or by maximum information gain) because the number of leaf nodes depends on the data and the splits made, not on the method of choosing them.

In the worst case, the tree could fit the training data perfectly by putting each training example in its own leaf node. If there are m training examples, the maximum number of leaf nodes is:

Maximum number of leaf nodes = m

This holds true whether splits are chosen randomly or using maximum information gain.

- (b) (2 pts) What is the maximum number of leaf nodes that a decision tree could contain if it were trained on m training examples using the original maximum mutual information version of the algorithm? Is it bigger, smaller, or the same as your answer to (b)?

The maximum number of leaf nodes that a decision tree could contain using the original maximum mutual information algorithm is the same as in the case of random splits. The reasoning is as follows:

In both cases, the structure of the tree depends on the data and the splits made, not on whether the splits are chosen based on maximum mutual information or randomly among valid options. In the worst-case scenario, the tree can perfectly fit the training data, isolating each training example into its own leaf node.

So, the maximum number of leaf nodes is:

Maximum number of leaf nodes = m

This is the same as the answer for random splits because the method of selecting splits does not affect the theoretical maximum number of leaf nodes.

- (c) (2 pts) How do you think this change (using random splits vs. maximum information mutual information splits) would affect the testing accuracy of the decision trees produced on average? Why?

If we use random splits instead of splits based on maximum mutual information, the testing accuracy of decision trees will probably go down because

1. Less Informative Splits: Splitting based on maximum mutual information picks the best option to reduce uncertainty (entropy) in the data. Random splits don't focus on this, so they might not divide the data as effectively, making the decision tree less accurate.

2. Suboptimal Tree Structure: Random splits can lead to a tree with more nodes or a bigger depth just to reach the same training accuracy. This can make the tree inefficient and more likely to overfit or underfit, which hurts testing accuracy.

3. testing Accuracy Drops: Random splits don't guarantee that the tree captures the most important patterns in the data, making it less reliable on new, unseen data.

On average decision trees made with random splits are likely to be less accurate on test data because their splits don't reduce uncertainty or highlight the most important parts of the data effectively.

5. (8 pts) Consider the following training set:

A	B	C	Y
0	1	1	0
1	1	1	0
0	0	0	0
1	1	0	1
0	1	0	1
1	0	1	1

Learn a decision tree from the training set shown above using the information gain criterion. Show your steps, including the calculation of information gain (you can skip $H(y)$ and just compute $H(y|\mathbf{x})$) of different candidate tests. You can randomly break ties (or better, choose the one that give you smaller tree if you do a bit look ahead for this problem).

Calculating the conditional entropy for each feature:

Split on A: A = 0: Rows [1, 3, 5] (Y = [0, 0, 1])

$$H(Y | A = 0) = - \left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3} \right) = 0.918$$

A = 1: Rows [2, 4, 6] (Y = [0, 1, 1])

$$H(Y | A = 1) = - \left(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3} \right) = 0.918$$

$$H(Y | A) = \frac{1}{2} \times 0.918 + \frac{1}{2} \times 0.918 = 0.918$$

Split on B: B = 0: Rows [3, 6] (Y = [0, 1]).

$$H(Y | B = 0) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1.0$$

B = 1: Rows [1, 2, 4, 5] (Y = [0, 0, 1, 1]).

$$H(Y | B = 1) = - \left(\frac{2}{4} \log_2 \frac{2}{4} + \frac{2}{4} \log_2 \frac{2}{4} \right) = 1.0$$

$$H(Y | B) = \frac{2}{6} \times 1.0 + \frac{4}{6} \times 1.0 = 1.0$$

Split on C: C = 0: Rows [3, 4, 5] (Y = [0, 1, 1]).

$$H(Y | C = 0) = - \left(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3} \right) = 0.918$$

C = 1: Rows [1, 2, 6] (Y = [0, 0, 1]).

$$H(Y | C = 1) = - \left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3} \right) = 0.918$$

$$H(Y | C) = \frac{1}{2} \times 0.918 + \frac{1}{2} \times 0.918 = 0.918$$

Choosing the best split among three : $H(Y | A) = 0.918$, $H(Y | B) = 1.0$, $H(Y | C) = 0.918$

We can pick A (or C) so let's choose A.

Split on A

For A = 0: Rows [1, 3, 5], with Y = [0, 0, 1].

Evaluate possible splits on B and C:

- Split on C gives pure subsets:
 - C = 0: Y = 1 (Row 5).
 - C = 1: Y = 0 (Rows 1, 3).
- Stop splitting here as both subsets are pure.

For A = 1: Rows [2, 4, 6], with Y = [0, 1, 1].

Evaluate possible splits on B and C:

- Split on B gives pure subsets:
 - B = 0: Y = 1 (Row 6).
 - B = 1: Further split on C:
 - * C = 0: Y = 1 (Row 4).
 - * C = 1: Y = 0 (Row 2).

Final Decision Tree:

- *Root: Split on A.*
 - *A = 0: Split on C.*
 - * *C = 0: Y = 1.*
 - * *C = 1: Y = 0.*
 - *A = 1: Split on B.*
 - * *B = 0: Y = 1.*
 - * *B = 1: Split on C.*
 - *C = 0: Y = 1.*
 - *C = 1: Y = 0.*

6. (8pts) Prove that

$$H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$$

Hint: you should start with the definition $H(X, Y) = -\sum_{x,y} P(x, y) \log P(x, y)$. Here we use X, Y to denote the random variables and x, y denote the values X and Y take, $P(x, y)$ is a short hand notation denoting $P(X = x, Y = y)$.

To prove that

$$H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y),$$

By the definition of the joint entropy:

$$H(X, Y) = -\sum_{x,y} P(x, y) \log P(x, y).$$

By the chain rule of probability, we know that $P(x, y) = P(x)P(y|x)$. Substituting this into the definition of $H(X, Y)$:

$$H(X, Y) = -\sum_{x,y} P(x, y) \log (P(x)P(y|x)).$$

$$H(X, Y) = -\sum_{x,y} P(x, y) (\log P(x) + \log P(y|x)).$$

Separating the terms and distributing the summation

$$H(X, Y) = -\sum_{x,y} P(x, y) \log P(x) - \sum_{x,y} P(x, y) \log P(y|x).$$

$-\sum_{x,y} P(x, y) \log P(x)$ simplifies because $\sum_y P(x, y) = P(x)$:

$$-\sum_{x,y} P(x, y) \log P(x) = -\sum_x P(x) \log P(x) = H(X).$$

$-\sum_{x,y} P(x, y) \log P(y|x)$ is the definition of conditional entropy $H(Y|X)$:

$$-\sum_{x,y} P(x, y) \log P(y|x) = H(Y|X).$$

$$H(X, Y) = H(X) + H(Y|X).$$

Since $H(X, Y)$ is symmetric (the joint entropy of X and Y is the same regardless of their order), we can switch the roles of X and Y . By applying the same reasoning:

$$H(X, Y) = H(Y) + H(X|Y).$$

From the two equations:

$$H(X, Y) = H(X) + H(Y|X),$$

$$H(X, Y) = H(Y) + H(X|Y),$$

$$H(X) + H(Y|X) = H(Y) + H(X|Y).$$

$$H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$$